Lo que pasa es que Kubernetes

Guía completa de Kubernetes en lenguaje sencillo - Muestra gratuita (Capítulo 1)



Luis Felipe Ariza Vesga

Lo que pasa es que Kubernetes

Lo que pasa es que Kubernetes

Guía completa de Kubernetes en lenguaje sencillo - Muestra gratuita (Capítulo 1)

Luis Felipe Ariza Vesga

Lo que pasa es que Kubernetes
Guía completa de Kubernetes en lenguaje sencillo - Muestra gratuita (*Capítulo 1*)

Copyright © 2026 Luis Felipe Ariza Vesga.
Todos los derechos reservados.

La información de este libro se distribuye "tal cual", sin garantía. Aunque se han tomado todas las precauciones en su preparación, ni el autor ni la editorial asumen responsabilidad alguna por pérdidas o daños derivados directa o indirectamente de su contenido.

ISBN: 9798173650245

Un tributo al zambullidor colombiano (Podiceps andinus), que vivió en los humedales de la sabana de Bogotá y en el lago de Tota, y que se extinguió en 1977 sin que nadie estuviera viendo.

Índice general

Convenciones de este libro **1**
Tipografía y referencias 1
Las cuatro cajas de aviso 2
Sobre los laboratorios 3
Créditos y licencias de terceros 4

Prefacio **5**
De qué trata este libro 5
Qué NO es este libro 6
Una serie que rinde tributo 7
Cómo navegar este libro 8
Sobre los ejemplos de código 8
Cómo contactarnos 9
Agradecimientos 10

I Fundamentos de Kubernetes **11**

1 Contenedores: de donde vienen y que problema resuelven **13**
1.1 En mi máquina sí funciona 13
1.2 La máquina virtual: mandar la casa entera 14
1.3 Qué es un contenedor de verdad: namespaces y cgroups 15
1.4 La iniciativa de contenedores abierta (OCI): el día que todos se pusieron de acuerdo 17
1.5 Y entonces, ¿dónde entra Kubernetes? La interfaz Runtime del contenedor (CRI) 18
1.6 Resumen 19

Bibliografía 20

Convenciones de este libro

Cómo está escrito este libro: el tono, la tipografía, las cuatro cajas de aviso y qué son los laboratorios.

Antes de arrancar, media página sobre cómo está escrito este libro. Se lee en dos minutos y le ahorra confusiones después.

Este libro le habla de **usted**. No es formalidad ni distancia: es como se habla a alguien con respeto. Si usted es de otro país y le suena raro, léalo como un "tú".

Cada capítulo abre con una escena cotidiana. Esa escena no es adorno ni relleno para hacer bulto: **es** el concepto técnico contado de otra manera, y va a volver al final del capítulo, en el resumen, ya explicada con las palabras correctas. Si en algún momento no entiende una explicación técnica, devuélvase a la escena del comienzo: casi siempre la respuesta está ahí en versión sencilla.

También va a encontrar humor. El humor cumple una función concreta, que es bajarle la dificultad técnica al libro. Nunca reemplaza el rigor: después del chiste viene siempre la definición oficial, textual, con su enlace a la fuente.

Tipografía y referencias

Tipografía monoespaciada se usa para todo lo que usted teclearía o leería en una pantalla: nombres de objetos como `Pod` y `Deployment`, campos de YAML como `restartPolicy`, comandos como `kubectl describe pod`, y rutas como `/api/v1`.

Negrita marca la primera aparición de un término importante, y también las palabras que el examen suele usar textualmente. Cuando usted vea una palabra en negrita en medio de una definición, es porque esa palabra exacta importa.

Los **paréntesis angulares** marcan lo que usted debe reemplazar: en `kubectl logs <nombre-del-pod>`, usted escribe el nombre real y no escribe los paréntesis.

Las **citas de la documentación oficial** aparecen integradas en el texto, no como bloques aparte, y siempre llevan un enlace a la fuente la primera vez que se usan en el capítulo. Eso es intencional: usted debe poder verificar cualquier afirmación de este libro sin buscarla, haciendo clic.

Cada capítulo cierra con una **bibliografía** de al menos tres fuentes, todas oficiales y todas con enlace.

Las **figuras** se referencian en el texto y llevan pie explicativo. Si una figura aparece cerca de una explicación, es porque esa explicación se entiende mejor mirándola.

Las cuatro cajas de aviso

A lo largo del libro va a encontrar cuatro tipos de caja. Cada una significa algo distinto, y le conviene reconocerlas de un vistazo porque no todas tienen la misma urgencia.

> **Nota**
> Información complementaria que amplía lo que acaba de leer. Se puede saltar sin perder el hilo, pero suele contener un detalle técnico que conviene conocer. Ejemplo real de este libro: kube-proxy es opcional si su plugin de red implementa por su cuenta el reenvío para los Services.

> **Consejo**
> Un atajo, una regla mnemotécnica o una forma más rápida de hacer algo. Aquí viven las frases para memorizar, como "la sonda de vida mata, la de disponibilidad desconecta".

> **Advertencia**
> Un error frecuente, una confusión habitual o un comportamiento que sorprende. Si usted solo va a leer un tipo de caja antes de presentar el examen, que sean estas. Ejemplo: CrashLoopBackOff no es una fase del Pod.

 **Precaución**
Algo que puede causar daño real en un clúster de verdad: pérdida de datos, caída de servicio o un hueco de seguridad. Estas son pocas y son en serio. Ejemplo: en aprovisionamiento dinámico, la política de reclamación por defecto es Delete, así que borrar un PVC (PersistentVolumeClaim) borra el disco.

En resumen: **nota** amplía, **consejo** ayuda, **advertencia** corrige y **precaución** protege.

Sobre los laboratorios

Cada una de las cuatro partes cierra con un **laboratorio**, y conviene decir desde ya qué son y qué no son.

No son ejercicios de copiar y pegar. Cada laboratorio tiene un objetivo, unos pasos con puntos, y comprobaciones que leen el estado real del clúster. Usted no aprueba un paso porque corrió un comando: lo aprueba porque el clúster quedó como debía quedar. Si usted llega al mismo resultado por otro camino, el laboratorio lo acepta igual, porque lo que se está evaluando es el estado final y no su historial de comandos.

Todos los comandos de los laboratorios fueron ejecutados en un clúster real antes de publicarse, y las versiones están fijadas. Eso significa que si algo no le funciona, lo más probable es que haya una diferencia de entorno que vale la pena entender, y no que el libro esté inventando.

Un aviso importante: el KCNA es un examen de opción múltiple de 90 minutos. No lo van a sentar frente a una terminal. Entonces los laboratorios **no** son entrenamiento de memoria muscular para el examen: son la forma de convertir cada afirmación de este libro en algo que usted vió con sus propios ojos y que le puede servir para los exámenes prácticos de Kubernetes. Es distinto creer que un Pod sin plugin de red deja el nodo en NotReady, a haberlo visto pasar en su máquina.

Y sobre dónde correrlos: nunca en un clúster de producción.

Créditos y licencias de terceros

Este libro reproduce y traduce fragmentos de la documentación oficial de Kubernetes (https://kubernetes.io/docs/), publicada por The Kubernetes Authors bajo licencia Creative Commons Attribution 4.0 International (CC BY 4.0). El texto original está disponible en https://creativecommons.org/licenses/by/4.0/.

Las definiciones citadas textualmente aparecen entre comillas y enlazadas a la página de la que provienen. Las traducciones al español son propias de este libro y no han sido revisadas ni avaladas por The Kubernetes Authors.

Kubernetes es una marca registrada de The Linux Foundation. Los nombres de las certificaciones citadas pertenecen a la Linux Foundation y a la Cloud Native Computing Foundation. Este libro es material de estudio independiente y no está afiliado a esas organizaciones, ni cuenta con su respaldo o patrocinio.

Prefacio

Por qué existe este libro, qué cubre, qué no cubre, y cómo leerlo.

Este libro nació de una molestia.

La molestia es que casi todo el material para aprender Kubernetes está escrito como si el lector ya supiera Kubernetes. Se abre con un diagrama de siete cajas conectadas por flechas, se dice "el kubelet se comunica con el apiserver mediante el CRI" (afirmación que, dicho sea de paso, es falsa: el kubelet habla con el apiserver por HTTPS/REST; el CRI, la Container Runtime Interface (interfaz de runtime de contenedores), es la interfaz del kubelet con el runtime de contenedores, no con el apiserver), y se sigue de largo sin corregirla. Quien ya entendía, asiente sin notar el error. Quien no, se siente bruto y cierra la pestaña.

Y no es bruto. Es que nadie le contó la historia completa.

Este libro cuenta la historia. Empieza en una cocina prestada y termina en un ave que se extinguió porque nadie estaba viendo, y en el medio están las cuatro áreas del examen KCNA con sus pesos oficiales. Se habla de usted, sin corbata y sin bajarle el nivel a nada.

Porque esa es la apuesta de fondo: que la claridad y el rigor no pelean. Que un chiste bien puesto no le quita seriedad a una explicación, sino que le abre la puerta. Y que después de esa puerta uno puede poner la definición oficial, textual, con su enlace a la fuente.

De qué trata este libro

Este volumen cubre la certificación **KCNA**, Kubernetes and Cloud Native Associate, de la Cloud Native Computing Foundation y la Linux Foundation.

Está organizado exactamente sobre los cuatro dominios oficiales del examen, con sus pesos reales, tomados del currículo publicado por la CNCF y no de resúmenes de terceros:

Parte 1, Fundamentos de Kubernetes, 44%. Qué es un contenedor de verdad, cómo está armado un clúster, los objetos que usted usa todos los días, cómo se le habla a la API (Application Programming Interface), y cómo el scheduler decide dónde va cada Pod.

Parte 2, Orquestación de contenedores, 28%. El modelo de red y quién lo implementa, el almacenamiento que sobrevive al pod, la seguridad del orquestador, y el troubleshooting cuando nada arranca.

Parte 3, Entrega de aplicaciones cloud native, 16%. GitOps con sus cuatro principios formales, Helm, las tres siglas de CI y CD, y la depuración de aplicaciones que ya están corriendo.

Parte 4, Arquitectura cloud native, 12%. La definición oficial de cloud native, el escalado horizontal y vertical, la CNCF y sus niveles de madurez, y la observabilidad.

Un aviso importante sobre esos porcentajes: el currículo del KCNA **cambió a partir del 24 de noviembre de 2025**. Pasó de cinco dominios a cuatro, y la observabilidad quedó incluida dentro de Arquitectura Cloud Native. Cualquier material que usted encuentre con cinco dominios y con un dominio propio de observabilidad está describiendo el examen **anterior**.

Qué NO es este libro

Este libro **no** es una referencia de comandos. No va a encontrar aquí todas las opciones de `kubectl`, porque para eso está la documentación oficial y `kubectl --help`. Memorizar comandos no es lo que evalúa el examen KCNA.

No es un libro de administración de clústeres en producción. No se cubre instalar un clúster con kubeadm, actualizar versiones, ni operar etcd. Eso es territorio del examen CKA.

No es un libro de desarrollo de aplicaciones. No se cubren patrones de microservicios, ni cómo escribir su aplicación. Eso lo encuentra en el examen CKAD.

No es un simulador de examen. Hay preguntas de práctica en el apéndice C, pero este libro se escribió para que usted **entienda**, no para que reconozca respuestas. Si lo que busca son bancos de preguntas, este no es el libro, y se lo digo de frente para no gastarle su plata.

Y **no** es el volumen de seguridad. El examen KCSA tiene su propio volumen, y todo lo que aquí se toca de seguridad (RBAC (Role-Based Access Control), ServiceAccounts, Pod Security Standards) está en el nivel que el examen KCNA exige, no en el que exige el examen KCSA.

Tampoco es un libro sobre el humor, aunque hable así. El tono es una herramienta pedagógica, no el tema.

Una serie que rinde tributo

Cada libro de la serie **Lo que pasa es que** lleva en su portada un animal que se extinguió por actividad humana, con una dedicatoria breve. No es decoración: es un tributo, y la memoria de que alguna vez vivió aquí con nosotros. A este volumen le tocó el zambullidor colombiano, un ave de los humedales de la sabana de Bogotá y del lago de Tota.

Este libro es el primer paso de un camino más largo. Según la página oficial del programa Kubestronaut, la CNCF (Cloud Native Computing Foundation) otorga el título de **Kubestronaut** a quien aprueba todas sus certificaciones de Kubernetes, que hoy son cinco: KCNA, KCSA, CKA, CKAD y CKS. Este volumen cubre el examen KCNA. Después vienen tres libros más de la serie: **Lo que pasa es que administrar Kubernetes** (CKA), **Lo que pasa es que desarrollar en Kubernetes** (CKAD) y **Lo que pasa es que asegurar Kubernetes** (CKS).

En ese camino, los tributos de la serie pasan por animales del aire, del agua y de la tierra. Así que para llegar a Kubestronaut usted no solo aprende a administrar un clúster, a desarrollar aplicaciones sobre él y a asegurarlo: también recorre, libro por libro, la memoria de especies del aire, del agua y de la tierra que ya no están con nosotros.

Cómo navegar este libro

Los trece capítulos están pensados para leerse en orden, porque cada uno se apoya en el anterior. El contenedor del capítulo uno vuelve como Pod en el tres, como contexto de seguridad en el ocho y como contenedor efímero en el once. Si usted salta, va a encontrar referencias a cosas que no leyó.

Dicho eso, la estructura le permite atacar por partes: cada una de las cuatro corresponde a un dominio del examen, así que si usted ya trabaja con Kubernetes y quiere reforzar solo un dominio, puede entrar directo a esa parte.

Cada capítulo tiene la misma forma. Abre con una escena, que casi siempre es una situación cotidiana, y esa escena **es** el concepto técnico, no un adorno. Después viene la explicación, apoyada en documentación oficial y con enlaces a la fuente en el primer uso. En el medio hay figuras y cajas de aviso. Y cierra con un resumen que retoma la escena inicial y la explica ya con el vocabulario aprendido.

Los resúmenes están escritos para ser releídos solos, el día antes del examen, sin volver al capítulo entero.

Al final del volumen hay tres apéndices: un **glosario** con los términos que el examen usa, una **lista de temas y trampas** con la que más se va a enfrentar, y unas **preguntas frecuentes** sobre la logística del examen y sobre las trampas conceptuales más comunes.

Y cada parte tiene un **laboratorio**. Los laboratorios no son opcionales ni decorativos: son donde usted comprueba en un clúster real que lo que leyó es cierto.

Sobre los ejemplos de código

Los ejemplos de código de este libro son fragmentos de YAML de Kubernetes y comandos de `kubectl`, `helm` y utilidades del sistema.

Se muestran en tipografía monoespaciada, así: `kubectl get pods`. Cuando un comando lleva partes que usted debe reemplazar, van entre paréntesis angulares, como en `kubectl logs <nombre-del-pod>`. No escriba los paréntesis.

Todos los comandos de los laboratorios fueron ejecutados en un clúster real antes de publicarse. No hay comandos escritos de memoria ni "esto debería funcionar": si un

comando está en un laboratorio, alguien lo corrió y vio su salida.

Las versiones importan en Kubernetes. Los laboratorios fijan las versiones que usan, y el texto indica la versión cuando un comportamiento depende de ella, como en el requisito de la API v1 del CRI desde Kubernetes 1.26. Si usted corre una versión distinta, algo puede cambiar, y por eso siempre se le enlaza la documentación oficial.

Todo el código de los laboratorios está publicado en https://github.com/lfarizav/lo-que-pasa-es-que-kubernetes. Ahí encuentra los manifiestos de YAML completos, los comandos de cada laboratorio y las versiones exactas con las que se probaron, listos para clonar y ejecutar en una terminal de Linux:

```
git clone https://github.com/lfarizav/lo-que-pasa-es-qu
 ↪  e-kubernetes.git
cd lo-que-pasa-es-que-kubernetes
```

Si un comando del libro no le funciona, revise primero ese repositorio: cuando la documentación oficial cambia, la corrección llega ahí antes que a la siguiente edición impresa.

Puede usar el código de este libro en sus programas y su documentación sin pedir permiso, salvo que esté reproduciendo una porción significativa. Vender o distribuir los ejemplos sí requiere permiso.

Y una advertencia práctica: nunca corra en un clúster de producción un comando de un libro, de un blog o de un video sin entender qué hace. Este libro incluido.

Cómo contactarnos

Este libro es parte de un catálogo de libros técnicos escritos por Luis Felipe Ariza Vesga, y se mantiene de forma continua: cuando el currículo del examen cambia, o cuando la documentación oficial cambia una definición, el libro se actualiza, se informa a los lectores y se entrega gratuitamente la nueva versión del libro.

Si usted encuentra un error, tiene una duda sobre una explicación, o cree que algo quedó mal contado, escriba. Los reportes de error son bienvenidos y son la forma más rápida de mejorar el material, sobre todo los que dicen "esta parte la leí tres veces y no la entendí", porque eso casi siempre significa que está mal escrita y no que usted esté lento.

Interesa especialmente saber dos cosas. La primera, si alguna explicación quedó desactualizada frente a la documentación oficial, con el enlace a la fuente que la contradice. La segunda, si algún comando de un laboratorio no funcionó en su entorno, con la versión de Kubernetes y del runtime que usted está usando.

Para la certificación en sí, la fuente autoritativa siempre es la Linux Foundation y la CNCF: ellos publican el currículo, los cambios de programa, la logística del examen y las políticas de reintento. Ningún libro, incluido este, reemplaza esa fuente. Antes de presentar, revise la página oficial del examen.

Agradecimientos

A la comunidad cloud native, que escribió toda la documentación en la que este libro se apoya y la dejó abierta y gratis para cualquiera. Cada dato de este volumen salió de ahí.

A los mantenedores de Kubernetes, de la OCI (Open Container Initiative), de OpenTelemetry, de Helm y de OpenGitOps, que además de construir el software se tomaron el trabajo de versionar sus definiciones y publicar sus cambios de programa.

A los estudiantes que en algún momento levantaron la mano para decir "profe, no entendí". Ese gesto, que cuesta, es el que hace que un material mejore. Buena parte de las escenas de este libro existen porque alguien dijo que la explicación anterior no le había servido.

Y finalmente al zambullidor colombiano, que vivió en los humedales de la sabana de Bogotá y en el lago de Tota, y del que se vieron los últimos dos o tres individuos en 1977. Este libro habla de eso en el último capítulo, y su portada lo recuerda.

PART I

Fundamentos de Kubernetes

1

Contenedores: de donde vienen y que problema resuelven

Por qué existe el contenedor, qué es de verdad por dentro, y por qué Kubernetes necesita uno para poder trabajar.

1.1 En mi máquina sí funciona

Imagínese que usted prepara un sancocho espectacular en su casa. Le queda perfecto, de esos que la gente recuerda. Invita a la familia, todo el mundo feliz, ahí en la cocina de su abuela, donde desde hace años cuelga en la pared un cuadro con la foto de un pájaro raro; ella nomás decía, cortico y en serio, que había que fijarse en las detalles mijito. Y entonces su tía le dice: "ome, hágame ese mismo sancocho en mi casa el domingo".

Usted llega confiado, con la misma receta y las mismas manos, y resulta que la estufa es de otra marca, la olla es más pequeña, el fogón calienta distinto, el agua sabe diferente y en esa cocina no hay guascas. Mismo cocinero. Misma receta. Y le queda horrible.

Bueno, mire: eso mismo le lleva pasando al software desde que el software existe. Usted escribe un programa, lo prueba en su computador, funciona perfecto. Lo manda

al servidor de la empresa y se cae. Y siempre, siempre, aparece la misma frase, dicha con la misma cara de inocencia: "pero es que en mi máquina sí funciona".

Lo que pasa es que el problema nunca fue el programa. El problema era la cocina. La versión de la librería, la variable de entorno que usted tenía configurada y se le olvidó, el archivo que estaba en su disco y en ningún otro lado, la versión del sistema operativo. Todo eso es la cocina. Y usted estaba entregando la receta sin entregar la cocina.

Un contenedor es, exactamente, la solución a ese problema: empacar la cocina junto con la receta. Eso es todo.

Entonces, cuando alguien le diga que un contenedor es "una máquina virtual liviana", usted ya va a saber que no. No no no no. Es otra cosa, y la diferencia le va a aparecer en el examen, porque la containerización es una de las competencias declaradas en el currículo oficial del examen KCNA. En este capítulo vamos a ver qué es de verdad un contenedor, por qué apareció, y por qué Kubernetes no podría existir sin eso. ¿Listo? Hágale mijito, decía mi abuelita.

1.2 La máquina virtual: mandar la casa entera

Antes de los contenedores la industria ya había intentado resolver esto, y la respuesta se llamaba máquina virtual. La idea era razonable: si el problema es que la cocina de allá es distinta a la de acá, pues mandemos la cocina entera. Una máquina virtual empaca un sistema operativo completo, con su propio kernel, sus propios drivers y su propio arranque, y lo corre encima de un hipervisor.

Y funcionó. De verdad funcionó, y buena parte de la nube que usted usa hoy se construyó encima de eso. Pero tenía un costo, y el costo era el peso. Cada máquina virtual arrastra un sistema operativo completo: gigabytes de disco, cientos de megabytes de memoria solo para existir, y un arranque que se mide en minutos, no en segundos. Si usted quiere correr veinte servicios pequeños, termina con veinte sistemas operativos completos peleando por la misma máquina física. Es como si, para llevarle el sancocho a su tía, usted se llevara la casa.

Entonces alguien hizo la pregunta correcta. Y la pregunta correcta casi siempre es una pregunta incómoda: ¿de verdad necesitamos otro kernel?

Porque piénselo. Si el servidor de allá y el computador de acá ya corren Linux, el kernel ya está puesto. Lo que cambia entre las dos máquinas no es el kernel: son las librerías, los archivos, las versiones, las variables de entorno. Eso es lo que hay que empacar. El kernel se puede compartir, hermano, ya está ahí.

Y de esa pregunta incómoda nació la idea del contenedor: compartir el kernel de la máquina anfitriona, pero darle a cada proceso la ilusión de que el sistema es de él solo. No es una máquina virtual liviana. Es una cosa completamente distinta, construida sobre una idea distinta.

1.3 Qué es un contenedor de verdad: namespaces y cgroups

Mire, aquí es donde mucha gente se pierde, y donde el examen le puede poner una trampa. Un contenedor no es un objeto. En Linux no existe una cosa llamada "contenedor" que usted pueda señalar con el dedo. Un contenedor es un proceso normal y corriente, del mismo tipo que cualquier otro proceso de su máquina, al que el kernel le está mintiendo de dos maneras al mismo tiempo.

La primera mentira se llama namespaces, y es una mentira sobre lo que el proceso puede ver. El kernel de Linux puede darle a un proceso su propia vista aislada de ciertos recursos del sistema: su propia lista de procesos, de modo que el proceso se cree el número uno y no vea a nadie más; su propia interfaz de red, con sus propias direcciones; su propio árbol de sistema de archivos; su propio nombre de host; sus propios usuarios. El proceso mira a su alrededor y honestamente cree que está solo en la máquina. No está solo. Simplemente no puede ver a los demás.

La segunda mentira se llama cgroups, abreviatura de control groups, y es una mentira sobre lo que el proceso puede consumir. Los namespaces aíslan la vista, pero no reparten los recursos: un proceso aislado que se enloquezca puede comerse toda la memoria y toda la CPU de la máquina y tumbar a todos los vecinos. Los cgroups le ponen el límite: este proceso puede usar tanta memoria y tanta CPU, y ni un gramo más.

Namespaces para lo que ve, cgroups para lo que gasta. La comparación completa contra la máquina virtual está en la Figura 1.1. Esas dos cosas juntas, aplicadas a un proceso normal, son un contenedor.

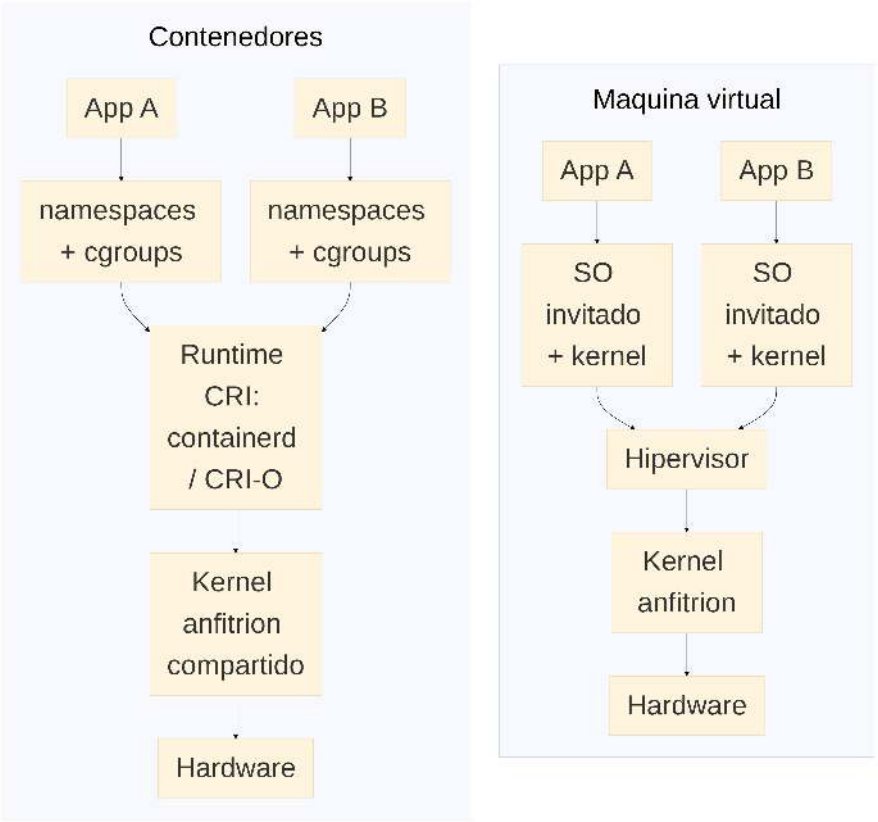


Figura 1.1. Cada máquina virtual arranca su propio kernel invitado sobre un hipervisor; los contenedores comparten el kernel del anfitrión y se aíslan con namespaces y cgroups.

Y ahí está la razón de fondo por la que un contenedor arranca en milisegundos y una máquina virtual en minutos: el contenedor no arranca ningún sistema operativo, porque el sistema operativo ya estaba prendido. ¿Sí o no?

> **Advertencia**
> Un contenedor no es una máquina virtual liviana. Una máquina virtual arranca su propio kernel invitado sobre un hipervisor. Un contenedor no arranca ningún kernel: es un proceso del anfitrión, aislado con namespaces y limitado con cgroups, sobre el kernel que ya estaba corriendo.

1.4 La iniciativa de contenedores abierta (OCI): el día que todos se pusieron de acuerdo

Ahora, tener la tecnología no basta. Durante los primeros años cada quien empacaba los contenedores a su manera, y eso, hermano, es la receta perfecta para que un ecosistema se parta en pedazos. Si la imagen que produce una herramienta solo la puede correr esa misma herramienta, usted no tiene un estándar: tiene una jaula con logotipo bonito.

Por eso el 22 de junio de 2015 se lanzó la Open Container Initiative, la OCI, formada por Docker, CoreOS y otros actores de la industria de contenedores, bajo el paraguas de la Linux Foundation. Es una estructura de gobernanza abierta y liviana, y su propósito declarado es crear estándares abiertos de industria alrededor de los formatos y los runtimes de contenedores. El objetivo era de una simpleza brutal: ponerse de acuerdo, para que una imagen construida con cualquier herramienta se pueda correr con cualquier otra.

La OCI mantiene tres especificaciones, y a usted le conviene sabérselas por nombre, porque en el examen aparecen así:

La **runtime-spec** describe cómo se ejecuta un "filesystem bundle", es decir, un paquete ya desempacado en disco y listo para correr.

La **image-spec** define el formato de imagen OCI: el manifiesto de la imagen, la serialización de las capas del sistema de archivos y la configuración necesaria para arrancar la aplicación.

La **distribution-spec** estandariza la API (Application Programming Interface) para distribuir imágenes, y llegó a la versión 1.0 en mayo de 2020. Está diseñada de forma tan genérica que sirve para distribuir cualquier tipo de contenido, no solamente

contenedores.

Las tres juntas producen ese resultado que usted da por sentado todos los días: usted descarga una imagen, se desempaca en un bundle, y se ejecuta con unos pocos argumentos. Sin tener que preguntarle a nadie quién la construyó ni con qué.

1.5 Y entonces, ¿dónde entra Kubernetes? La interfaz Runtime del contenedor (CRI)

Aquí llegamos al punto donde este capítulo se conecta con todo el resto del libro. Kubernetes no corre contenedores.

Léalo otra vez, porque en el examen esto se pregunta de forma torcida: Kubernetes no corre contenedores. Kubernetes le pide a otro programa que los corra.

Ese otro programa se llama runtime de contenedores, y la conversación entre los dos ocurre a través de un protocolo llamado Container Runtime Interface, el CRI. Del lado de Kubernetes quien habla es el kubelet, el agente que corre en cada nodo. Del otro lado está el runtime. El CRI es, sencillamente, el idioma común entre esos dos.

Los runtimes que Kubernetes soporta oficialmente son containerd, CRI-O, Docker Engine y Mirantis Container Runtime. Y hay un detalle de versión que vale oro: desde Kubernetes 1.26 el proyecto solo funciona con la versión v1 de la API del CRI. Si un runtime no soporta la v1, el kubelet sencillamente no registra el nodo. No es que funcione a medias ni que quede lento: el nodo no aparece.

Hay una segunda trampa, más silenciosa, y tiene que ver con los cgroups que ya vimos. Tanto el kubelet como el runtime necesitan hablar con los cgroups para aplicar los límites de recursos de los pods y los contenedores. Es crítico que los dos usen el mismo driver de cgroups y estén configurados igual. Los dos drivers disponibles son cgroupfs, que es el valor por defecto del kubelet, y systemd, que es el recomendado cuando el sistema usa systemd como init y el recomendado también con cgroups v2. Si usted los deja distintos, no le va a explotar de una: le va a fallar raro y bajo presión de recursos, que es la peor forma de fallar que existe.

> **Precaución**
> El kubelet y el runtime deben usar el MISMO driver de cgroups. Si uno usa cgroupfs y el otro systemd, el nodo puede arrancar y verse sano, y fallar después bajo presión de recursos. Con cgroups v2 y un sistema con systemd como init, la recomendación oficial es systemd en ambos lados.

1.6 Resumen

Volvamos a la cocina de su tía. Es domingo, el sancocho ya está oliendo tan rico que a usted se le hace agua la boca, y aun así, como ya sabemos, le queda horrible. Ahora sí lo podemos explicar bien.

El problema de fondo nunca fue el programa, sino el entorno: las librerías, las versiones, las variables, los archivos. Las máquinas virtuales resolvieron eso mandando la cocina entera, con kernel propio incluido, y funcionaron de verdad, pero al precio de arrastrar un sistema operativo completo por cada servicio.

El contenedor hace la apuesta contraria: comparte el kernel de la máquina anfitriona y aísla solamente lo que hace falta aislar. Un contenedor no es un objeto, ni una máquina virtual liviana: es un proceso normal al que el kernel le aplica namespaces, que limitan lo que el proceso puede ver, y cgroups, que limitan lo que el proceso puede consumir. Por eso arranca en milisegundos y no en minutos.

Para que ese modelo no se partiera en pedazos propietarios, la industria creó la Open Container Initiative en junio de 2015, bajo la Linux Foundation, con tres especificaciones que le conviene recordar por nombre: runtime-spec para la ejecución, image-spec para el formato de imagen y distribution-spec para la distribución.

Y Kubernetes, que es el tema de todo el resto del libro, no corre contenedores por sí mismo: el kubelet le habla a un runtime a través del CRI. containerd, CRI-O, Docker Engine y Mirantis Container Runtime son los soportados oficialmente, desde la versión 1.26 solo con la API v1, y con el driver de cgroups configurado igual en ambos lados.

Ya tenemos la cocina empacada, y ahora sí la podemos llevar a cualquier lado: misma receta, mismo resultado, en la casa de su tía, en el servidor de la empresa, donde sea. El sancocho por fin le queda rico en todas partes. Aunque, hágale caso a esto,

hermano: el de su tía siempre le va a quedar más rico, porque a la receta ella le mete amor, y eso no lo empaca ningún contenedor.

Ahora sí podemos hablar de quién organiza las cocinas cuando hay cientos. Eso es Kubernetes, y es el próximo capítulo.

Bibliografía

- Kubernetes Documentation, Container Runtimes (production environment setup): supported runtimes, CRI v1 requirement, and cgroup driver configuration. https://kubernetes.io/docs/setup/production-environment/container-runtimes/
- Open Container Initiative, About the OCI: governance, founding, and the runtime, image and distribution specifications. https://opencontainers.org/about/overview/
- Kubernetes Blog, Upcoming changes in Kubernetes 1.26: removal of the CRI v1alpha2 API and the resulting v1-only requirement. https://kubernetes.io/blog/2022/11/18/upcoming-changes-in-kubernetes-1-26/
- CNCF, Kubernetes and Cloud Native Associate (KCNA) Exam Curriculum: domain weights and competencies, Containerization under Kubernetes Fundamentals. https://github.com/cncf/curriculum